

---

```
>>> bacon = 'Goodbye'
>>> id(bacon)
33827584
❶ >>> bacon = 'Hello'
>>> id(bacon)
33863820
❷ >>> bacon = bacon + ', world!'
>>> bacon
'Hello, world!'
>>> id(bacon)
33870056
❸ >>> bacon[0] = 'J'
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'str' object does not support item assignment
```

---

Strings are immutable, so you cannot change their value. Although it looks like the string's value in `bacon` is being changed from 'Goodbye' to 'Hello' ❶, it's actually being overwritten by a new string object with a new identity. Similarly, an expression using string concatenation creates a new string object ❷ with a new identity. Attempting to modify the string in-place with item assignment isn't allowed in Python ❸.

A tuple's value is defined as the objects it contains and the order of those objects. *Tuples* are immutable sequence objects that enclose values in parentheses. This means that items in a tuple can't be overwritten:

---

```
>>> eggs = ('cat', 'dog', [2, 4, 6])
>>> id(eggs)
39560896
>>> id(eggs[2])
40654152
>>> eggs[2] = eggs[2] + [8, 10]
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'tuple' object does not support item assignment
```

---

But a mutable list inside an immutable tuple can still be modified in-place:

---

```
>>> eggs[2].append(8)
>>> eggs[2].append(10)
```